

Concrete Compressive Strength Prediction Using Machine Learning

Abstract

The objective of this work is to design and evaluate a machine learning pipeline capable of predicting the compressive strength of concrete based on its age and constituent ingredients. The dataset used, known as Concrete, contains 1,030 samples and nine variables, of which eight represent input features and one represents the target variable, compressive strength. This study includes exploratory data analysis, preprocessing steps, model selection, and hyperparameter tuning. Several regression algorithms were evaluated, and the best-performing model was identified and saved for future reproducibility.

Introduction

Concrete compressive strength is a critical parameter in civil engineering, directly linked to the quality, durability, and safety of construction projects. Predicting this strength based on mixture composition and curing age can reduce costs and accelerate the design and production process.

The dataset used in this work consists of eight features represent cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate, and age, while the target variable is the concrete compressive strength measured in megapascals (MPa).

The aim of this study is threefold: Perform data analysis and preprocessing, train and compare different regression models and provide a reproducible framework for future predictions.

Dataset Description

First, it is essential to determine the type of dataset at hand in order to select the most appropriate models. To this end, I developed functions designed to describe the dataset, identify variable classes, and calculate their means, medians, and standard deviations. For example, descriptive statistics in Table 1 below, show that the compressive strength ranges from a minimum of approximately 2.33 MPa to a maximum of 82.60 MPa, with a mean of 35.82 MPa. This relatively wide distribution indicates that the dataset covers a diverse set of concrete mixtures.

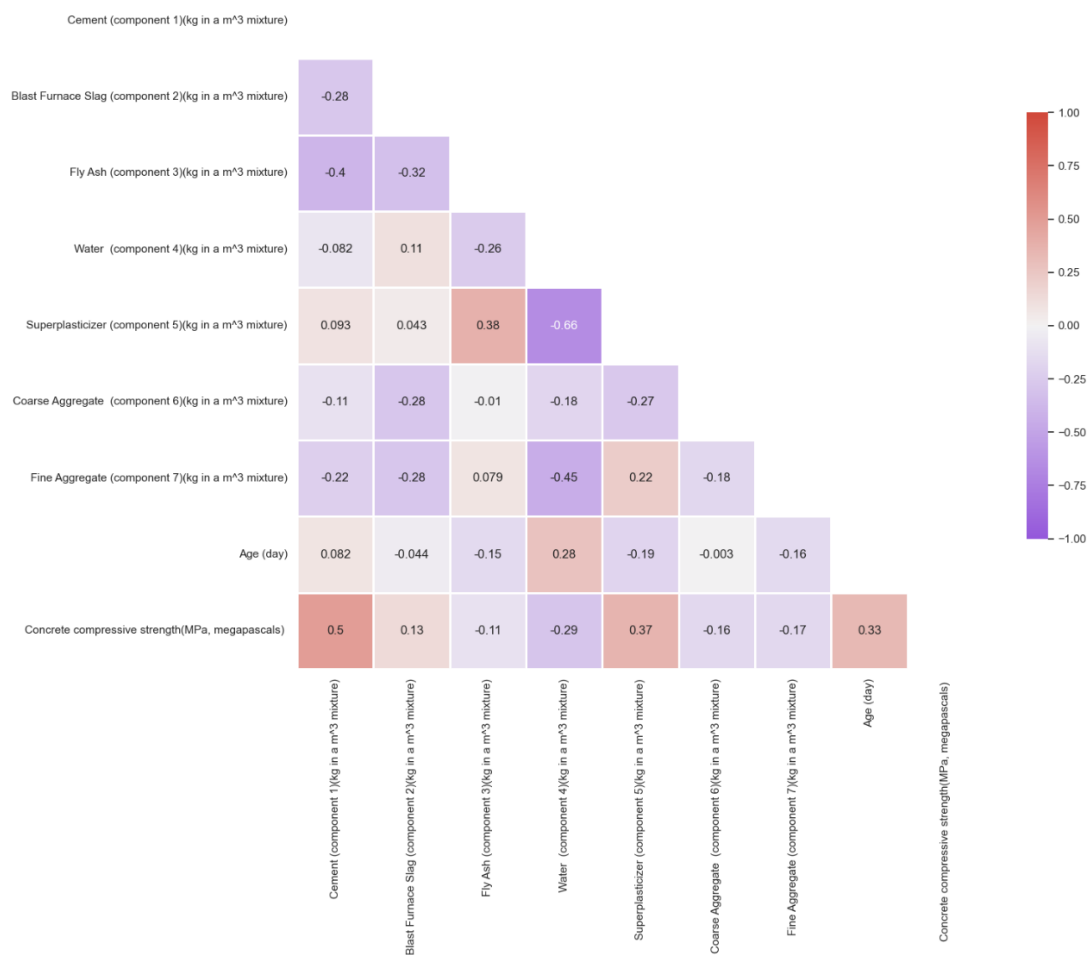
Table 1: Descriptive statistics of the dataset Concrete

	count	mean	std	min	25%	50%	75%	max
Cement (component 1)(kg in a m ³ mixture)	1030.0	281.165631	104.507142	102.000000	192.375000	272.900000	350.000000	540.000000
Blast Furnace Slag (component 2)(kg in a m ³ mixture)	1030.0	73.895485	86.279104	0.000000	0.000000	22.000000	142.950000	359.400000
Fly Ash (component 3)(kg in a m ³ mixture)	1030.0	54.187136	63.996469	0.000000	0.000000	0.000000	118.270000	200.100000
Water (component 4)(kg in a m ³ mixture)	1030.0	181.566359	21.355567	121.750000	164.900000	185.000000	192.000000	247.000000
Superplasticizer (component 5)(kg in a m ³ mixture)	1030.0	6.203112	5.973492	0.000000	0.000000	6.350000	10.160000	32.200000
Coarse Aggregate (component 6)(kg in a m ³ mixture)	1030.0	972.918592	77.753818	801.000000	932.000000	968.000000	1029.400000	1145.000000
Fine Aggregate (component 7)(kg in a m ³ mixture)	1030.0	773.578883	80.175427	594.000000	730.950000	779.510000	824.000000	992.600000
Age (day)	1030.0	45.662136	63.169912	1.000000	7.000000	28.000000	56.000000	365.000000
Concrete compressive strength(MPa, megapascals)	1030.0	35.817836	16.705679	2.331808	23.707115	34.442774	46.136287	82.599225

The various functions used enabled me to observe that the target variable we seek to predict (concrete compressive strength) is a numerical value, and thus classification models will not be considered. Additionally, it was crucial to identify the correlations between the various explanatory variables. The heatmap highlights that there is no excessively high correlation among my variables, which could otherwise lead to multicollinearity issues and bias the prediction results.

In the heatmap below, the strongest correlations are observed between water and superplasticizer, with a negative correlation of -0.66, and between concrete compressive strength and cement, with a positive correlation of 0.5. However, these correlations do not appear to be sufficiently high to significantly bias the prediction results.

Graph 1: Heatmap of correlation between variables



The second step of the treatment process was to identify missing values and outliers that needed to be addressed. Since my dataset had already been preprocessed, there were no missing values. However, boxplots and the z-score metric revealed several of outliers. The question was whether these values genuinely represented abnormal or unusual observations or were simply extreme yet relevant values in the analysis.

I concluded that these values did not require any modification or removal. For instance, the "age" variable contains 33 outliers, but these are not outliers in the strict sense; rather, most data points are concentrated at 28 days, while retaining these values remains relevant because concrete properties evolve over time. Similarly, the majority of the other outliers appear realistic but are simply farther from the mean; they still represent meaningful combinations worth studying. Removing these values could even constitute an ethical error, as they reflect possible observations or mixtures.

Split and scaling the data

Now that the dataset has been processed, we prepare it to facilitate its use by machine learning models. To train the models and ensure the reproducibility of our best predictive model, we split the initial dataset into three subsets: A training set, which contains 70% of the initial data and is used to train the learning algorithms. A validation set, which contains 15% of the initial data and is used to evaluate and fine-tune the model during training. A test set, which contains the remaining 15% of the initial data and is used to provide an unbiased evaluation of the final model, after training and validation.

This step is crucial because, to properly assess the robustness of our models, it is essential to evaluate them on data they have never seen and that were not used during training. Otherwise, the models risk overfitting and becoming unusable in real-world applications.

This process results in four datasets in total: the initial dataset, and the three derived subsets, with 1,030, 721, 154, and 155 observations respectively. Each of these subsets is divided into explanatory variables and the dependent variable to be predicted, so that the models are trained only on the explanatory variables, while the target variable Y is retained for comparison.

A second important step is scaling the data. By bringing all variables to the same scale, we prevent features with large numerical values from disproportionately influencing the learning process and dominating the predictions, even when their effect is not inherently more important. Scaling also ensures that differences in measurement units across variables are properly handled.

In practice, the scaling procedure must be applied carefully to avoid data leakage. Specifically, the scaler is fitted only on the training set (*fit_transform*), and then the same

transformation parameters (mean and standard deviation) are applied to the validation and test sets (*transform*). This ensures that information from the validation or test sets does not leak into the training process, which would otherwise bias the evaluation of the model's performance.

In addition, I added a step to save the scaler (fitted on the training set). This allows me to reuse the exact same transformation for any new incoming data.

Fitting the models

The model fitting step is what allows us to find the best way to make predictions on our data. There are many approaches to fitting models. In this project, I used a simple linear regression model, a k-nearest neighbors model, a decision tree, a random forest, a gradient boosting tree, and XGBoost. The main point is to train the model on the training dataset with the training predict variable.

Simple linear regression allows us to understand the impact of each explanatory variable on the dependent variable and then use this relationship to make predictions. In our case, we obtain an equation of the following form:

$$Y = 35.88 + 11.06 * \textit{Ciment} + 7.45 * \textit{Blast Furnace Slag} + 4.70 * \textit{Fly Ash} + -4.01 * \textit{Water} + 2.00 * \textit{Superplasticizer} + 0.43 * \textit{Coarse Aggregate} + 0.30 * \textit{Fine Aggregate} + 7.28 * \textit{Age}$$

Otherwise, K-nearest neighbors (KNN) allow us to predict the target value based on the closest training examples in the feature space, making it a non-parametric method that adapts to local patterns. In this case, we use 5 neighbors by default.

Decision trees work by splitting the data step by step based on feature values. At each split, the data is divided into smaller groups that are more similar in terms of the target variable. This process continues until the model can make predictions for each group. Because we can follow the splits, decision trees are easy to understand and interpret. Our decision trees contain 18 depths.

Random forests combine multiple decision trees to improve predictive performance and reduce overfitting. Each tree is trained on a random subset of the data and features, which introduces diversity among the trees. The final prediction is obtained by averaging the outputs of all trees making the model more robust and less sensitive to noise in the training data. By default, there is 100 random forests generated.

Gradient boosting trees build an ensemble of trees sequentially, where each new tree corrects the errors of the previous ones, improving accuracy. Each tree is usually small and shallow for avoiding overfitting.

XGBoost is an optimized implementation of gradient boosting that is highly efficient, handles missing values well, and often delivers state-of-the-art performance in tabular data tasks.

Model evaluation

Now that we have trained our models on the training set, we evaluate their performance using the validation dataset by making predictions on it. We then compare the predicted values with the actual values from the validation set. To do this, we use metrics such as R^2 , RMSE, and MAE: R^2 (coefficient of determination) measures how well the model explains the variance in the data. RMSE (Root Mean Squared Error) calculates the square root of the average squared differences between predicted and actual values, giving higher weight to larger errors and MAE (Mean Absolute Error) that computes the average absolute difference between predicted and actual values, providing a straightforward measure of prediction accuracy.

In our case XBoost have the best R^2 (0.9254) and the lowest RMSE (4.508) et MAE (3.219) that's makes him the best model. An R^2 of 0.9254 means that the XGBoost model explains about 92.5% of the variance in the target variable. In other words, most of the variability in the data is captured by the model, indicating a very good fit. Combined with the lowest RMSE (4.508) and MAE (3.219), this shows that XGBoost not only explains the variance well but also makes accurate predictions with relatively small errors. Therefore, without any hyperparameter tuning, XGBoost emerges as the best-performing model for our dataset.

In this section, I also performed a prediction test on the training set to identify which models memorize the training data best and to highlight the importance of overfitting. This show that's Decision Tree and Random Forest are good for memorizing training examples.

Hyperparameter tuning

In this section, we focused on the three best-performing models, Random Forest, Gradient Boosting Tree, and XGBoost Regressor, to optimize their hyperparameters and compare their performance more accurately. Hyperparameter tuning is important because the choice of hyperparameters can significantly impact a model's ability to generalize, improving predictive accuracy and reducing overfitting.

There are different strategies for hyperparameter tuning: Grid Search that systematically explores all combinations of a predefined set of hyperparameter values, ensuring a thorough search but potentially requiring more computational resources. And Random Search that samples random combinations of hyperparameters, which can be faster and often finds good results with fewer iterations but does not guarantee that all combinations are evaluated.

For this project, we chose to use Grid Search exclusively, and all parameters were selected automatically to ensure a consistent and reproducible optimization process.

For the Random Forest model, I applied GridSearchCV to optimize its hyperparameters. Specifically, I tested different values for the number of trees (*n_estimators*), the maximum depth of each tree (*max_depth*), and the minimum number of samples required to split a node (*min_samples_split*). GridSearchCV systematically evaluated all possible combinations using a 10-fold cross-validation and selected the best configuration. The results showed that the optimal Random Forest used 200 trees, unlimited depth, and a minimum split of 2 samples, which provided the strongest performance on the training data.

Furthermore, for the Gradient Boosting model, I explored different values for the number of boosting stages, the learning rate which corresponds to the level of error correction, and the maximum depth of the trees. The optimal model was obtained with 200 estimators, a learning rate of 0.2, and a maximum depth of 5, which together provided the strongest predictive performance.

Finally, for the XGBoost model the optimal configuration was obtained with 200 estimators, a learning rate of 0.1, and a maximum depth of 5.

The R^2 , RMSE and MAE results obtained are as follows:

	R^2	RMSE	MAE
Random Forest	0.8992	5.1829	3.6157
Gradient Boosting	0.9462	3.4205	2.3613
XGB Regressor	0.9371	4.0964	2.7578

We can conclude that the best model after hyperparameter tuning is Gradient Boosting, achieving an R^2 of 0.94, an RMSE of 3.42, and a MAE of 2.36. Therefore, this model will be retained for the final evaluation on the test set.

Evaluation on the test set

Now that we have identified the best model, it is essential to test its robustness on the test set, which has never been used or modified during training or validation. This step ensures that the model's performance is not just the result of memorizing previous data, but that it can generalize well to truly unseen observations.

Then, the training and validation sets are first combined into a single dataset so that the final model can be trained on as much data as possible before testing. The chosen model (*Gradient Boosting*) is then refitted on this larger training dataset, and finally, predictions are generated on the untouched test set. This approach allows us to obtain a realistic and unbiased estimate of the model's predictive performance in practice.

By comparing the actual values of Y from the test set with the predictions made by the final model, we obtain a performance of $R^2 = 0.956$, $RMSE = 3.420$, and $MAE = 2.361$. As a result, this model will be retained for future predictions and applied to new incoming data.

Deploying the model

The final step of this machine learning project is to deploy the trained model for use on new data. To do this, we first save the final model. We then simulate the arrival of new data by taking the initial dataset without the target variable. The previously saved scaler is applied to these new data to ensure they are transformed in the same way as the training data. Finally, the saved model is loaded and applied to the scaled data, allowing us to generate robust predictions based on the best machine learning model according to this data.

Conclusion

In conclusion, this project highlighted the key steps involved in building a machine learning model, aimed, for example, at enabling companies or organizations to make robust predictions. To achieve this, we carried out data preprocessing and exploration, feature scaling, testing of different models, hyperparameter optimization, and performance evaluation, ultimately ensuring reproducibility and enabling the deployment of the best model. For this dataset, the most effective learning model was the Gradient Boosting model with 200 estimators, a learning rate of 0.2, and a maximum depth of 5, which together provided the strongest predictive performance.